

Chapter 3 — Types and Forms (`Services/crud.types.ts`)

File: `src/features/react-crud-v4/Services/crud.types.ts` **One job:** Define UI-layer types — the edit context shape and the form validation schema. These are NOT API shapes (those live in `crud.dtos.ts`).

3.1 Why a Separate `types.ts` File?

`crud.dtos.ts` holds shapes that match the backend 1:1. `crud.types.ts` holds shapes that are UI concerns — data structures that only exist in the frontend.

The rule: **if the backend doesn't know about it, it belongs in `types.ts`.**

- `ItemEditContext` — the backend has no concept of "edit context". It's a UI construct for pre-populating the modal.
 - `ItemFormSchema` — the backend has its own validation. The frontend has its own. They are separate concerns.
 - `ItemFormValues` — the shape of form data managed by React Hook Form. The backend never sees this exact shape.
-

3.2 The Real File — Line by Line

```
import { z } from "zod";
```

Zod is used here for the form validation schema — same library, different purpose. In `dtos.ts` it validates API responses. Here it validates form submissions.

`ItemEditContext`

```
export type ItemEditContext = {  
  mode: "create" | "update";  
  studentActivityId?: number;  
  studentId: number;  
  studentKey: string;  
  activityId: number;  
  completedDate: Date;  
  completedBy: number;  
  completedByCode?: string;  
  notes: string;  
  restrictedNotes: boolean;  
};
```

What is `ItemEditContext`? When the user clicks "Edit" on a row, the app needs to pre-populate the modal form with that row's data. `ItemEditContext` is the shape of that pre-populated data. It is built by `buildEditContext(dto)` and passed to `<ItemModal>`.

`mode: "create" | "update"` — this is a **discriminated union**. The same modal handles both creating and editing. The `mode` field tells the modal which mutation to call on submit.

This is a TypeScript best practice called a **discriminated union** — a union type where one field (the "discriminant") distinguishes between the variants. TypeScript uses it to narrow types:

```
// Inside ItemModal.tsx:
const isEdit = editCtx.mode === "update";

// TypeScript now knows:
// - if mode === "update": studentActivityId MIGHT exist (it's optional)
// - if mode === "create": studentActivityId definitely doesn't exist
```

`studentActivityId?: number` — the `?` makes this field optional. When creating, there is no existing activity ID yet. When editing, there is. The `?` means "this field may or may not be present". TypeScript will warn you if you try to use it without checking it exists first.

`studentId: number` — required in both create and update. When creating, this is the student you're recording an activity for. When updating, this identifies which student's activity to update.

`completedDate: Date` — a JavaScript `Date` object, not a string. The API sends ISO strings ("`2024-03-15T00:00:00`"). The mapper converts string → `Date`. The form's date picker works with `Date` objects. The service converts `Date` → string when building the request payload. Each layer uses the format most natural to it.

`completedBy: number` — the staff ID of who completed the activity. `0` is a sentinel value meaning "not yet set" — the form will look up the logged-in user.

`completedByCode?: string` — optional staff code used to pre-populate the `TeacherSearch` component when editing. Not needed when creating.

Why not just pass the `ItemDto` directly to the modal? The DTO has the API shape (PascalCase, ISO strings, raw IDs). The modal needs the UI shape (camelCase, `Date` objects, form-ready values). Passing the DTO directly would couple the modal to the API shape — if the API changes, the modal breaks. The `ItemEditContext` is a stable UI contract.

ItemFormSchema

```
export const ItemFormSchema = z.object({
  activityId: z
    .string({ required_error: "Activity is required" })
    .min(1, "Activity is required"),
  completedDate: z
```

```

    .date({ required_error: "Date is required" })
    .refine((d) => d <= new Date(), "Date cannot be in the future"),
  completedBy: z
    .number({ required_error: "Completed By is required" })
    .min(1, "Completed By is required"),
  notes: z.string().default(""),
  restrictedNotes: z.boolean().default(false),
});

```

What is `ItemFormSchema`? This is the validation schema for the form. When the user clicks "Save", React Hook Form runs this schema against the form values. If any field fails, the form shows error messages and does NOT call the API.

Why Zod for form validation? React Hook Form supports multiple validation strategies. We use Zod because:

1. The schema is the single source of truth for both validation rules AND the TypeScript type
2. Zod's error messages are customizable and consistent
3. The same Zod knowledge used for DTOs applies here — one library, two uses

`activityId: z.string({ required_error: "..." }).min(1, "...")`

Why is `activityId` a `string` in the form but a `number` in the DTO and request?

Shadcn's `<Select>` component returns string values. When the user selects "Activity 42", the select returns "42" (a string), not 42 (a number). The form stores it as a string. The modal converts it to a number before sending to the API: `activityId: Number(values.activityId)`.

Two error messages for one field:

- `required_error: "Activity is required"` — fires when the field is `undefined` (user never touched it)
- `.min(1, "Activity is required")` — fires when the field is `""` (empty string — user cleared it)

You need both because Zod distinguishes between "field is missing" and "field is empty". Without `required_error`, a never-touched field shows no error. Without `.min(1)`, a cleared field shows no error.

`completedDate: z.date({ required_error: "..." }).refine((d) => d <= new Date(), "...")`

`z.date()` validates that the value is a JavaScript `Date` object (not a string). The form's date picker stores dates as `Date` objects, so this matches.

`.refine((d) => d <= new Date(), "Date cannot be in the future")` — a custom validation rule.
`.refine` takes:

1. A function that returns `true` (valid) or `false` (invalid)
2. An error message to show when the function returns `false`

`d <= new Date()` — compares the selected date to today. If the user picks a future date, this returns `false` and the error message is shown.

`completedBy: z.number({ required_error: "..." }).min(1, "...")`

`completedBy` is a staff ID number. `0` is the sentinel "not set" value. `.min(1)` ensures the user has actually selected a staff member (any valid staff ID is ≥ 1).

`notes: z.string().default("")`

`.default("")` — if the field is absent when the form initializes, use `""` as the value. This prevents TypeScript from complaining about potentially `undefined` values when you do `form.watch("notes")`. Without `.default("")`, `notes` could be `string | undefined` — you'd have to null-check it everywhere.

`restrictedNotes: z.boolean().default(false)`

Same pattern — defaults to `false` so the checkbox starts unchecked.

ItemFormValues

```
export type ItemFormValues = z.infer<typeof ItemFormSchema>;
```

Same `z.infer` pattern as DTOs. The result is:

```
type ItemFormValues = {  
  activityId: string;  
  completedDate: Date;  
  completedBy: number;  
  notes: string;  
  restrictedNotes: boolean;  
}
```

React Hook Form uses this type to give you full TypeScript autocomplete:

```
const { watch } = useFormContext<ItemFormValues>();  
watch("activityId") // TypeScript knows this is a string ✓  
watch("typo")       // TypeScript error: "typo" is not a key of ItemFormValues ✓
```

3.3 How React Hook Form Works (The Big Picture)

React Hook Form (RHF) is a library that manages form state efficiently. Understanding how it works is essential for working with `ItemModal` and `ItemForm`.

The Core Concept: Uncontrolled Inputs

Traditional React forms use **controlled inputs** — every keystroke updates state, which triggers a re-render:

```
// Controlled – re-renders on every keystroke
const [name, setName] = useState("");


```

React Hook Form uses **uncontrolled inputs** — the form values are stored in a ref (not state), so typing does NOT trigger re-renders. The form only re-renders when validation state changes (errors appear/disappear).

Why does this matter? A form with 10 fields that re-renders on every keystroke = 10 re-renders per character typed. With RHF, typing in one field does not re-render the other 9 fields. This is a significant performance improvement.

The `useForm` Hook

```
const methods = useForm<ItemFormValues>({
  resolver: zodResolver(ItemFormSchema),
  defaultValues,
});
```

`useForm` creates a form instance. It returns an object with:

- `methods.control` — used by `Controller` to register fields
- `methods.handleSubmit(fn)` — wraps your submit function with validation
- `methods.reset(values)` — resets the form to new values
- `methods.watch("fieldName")` — subscribes to a field's value (triggers re-render on change)
- `methods.formState.errors` — validation errors for each field
- `methods.formState.submitCount` — how many times the form has been submitted

resolver: `zodResolver(ItemFormSchema)` — connects Zod to RHF. When `handleSubmit` is called, it runs `ItemFormSchema.parse(formValues)`. If validation fails, it populates `formState.errors`. If it passes, it calls your `onSubmit` function with the validated values.

defaultValues — the initial values for all form fields. Without this, fields start as `undefined`, which causes React to warn about "uncontrolled to controlled" component transitions.

FormProvider and useFormContext

The form instance (`methods`) needs to be accessible to `ItemForm` (a child component). There are two ways to do this:

Option 1 (bad): Prop drilling

```
// ItemModal passes methods as a prop
<ItemForm control={methods.control} errors={methods.formState.errors} />
// ItemForm receives it
function ItemForm({ control, errors }) { ... }
```

Problem: Every new field you add to the form requires updating the prop type.

Option 2 (good): **FormProvider** + **useFormContext**

```
// ItemModal wraps ItemForm in FormProvider
<FormProvider {...methods}>
  <ItemForm />
</FormProvider>

// ItemForm reads the form from context
function ItemForm() {
  const { control, formState: { errors } } = useFormContext<ItemFormValues>();
}
```

FormProvider puts the form instance into React Context. **useFormContext** reads it from context. No props needed. Adding a new field to **ItemForm** requires zero changes to **ItemModal**.

Controller for Custom Inputs

Standard HTML inputs (**<input>**, **<textarea>**) work with RHF's **register()**:

```
<input {...register("name")} />
```

Shadcn's custom components (**<Select>**, **<Checkbox>**, **<Calendar>**) are NOT standard HTML inputs. They don't have a **ref** that RHF can attach to. They use **value/onChange** props instead.

Controller bridges this gap:

```
<Controller
  name="activityId"
  control={control}
  render={({ field }) => (
    <Select value={field.value} onChange={field.onChange}>
      ...
    </Select>
  )}
/>
```

field contains:

- **field.value** — the current value from RHF's store
- **field.onChange** — call this when the value changes (RHF updates its store)
- **field.onBlur** — call this when the field loses focus (RHF marks field as "touched")
- **field.ref** — attach to the DOM element for focus management

3.4 The Validation Flow — Step by Step

1. User fills in the form
2. User clicks "Save Changes"
3. `form.handleSubmit(onSubmit)` is called
4. RHF collects all field values from its internal store
5. `zodResolver` runs `ItemFormSchema.parse(formValues)`
6. **If validation fails:**
 - Zod returns errors for each invalid field
 - RHF populates `formState.errors`
 - The component re-renders showing error messages
 - `onSubmit` is NOT called
7. **If validation passes:**
 - `onSubmit(validatedValues)` is called
 - `validatedValues` is typed as `ItemFormValues` — you can trust its types
 - The mutation is called with the validated data

3.5 Common Mistakes

✗ Forgetting `required_error` for fields that start as `undefined`

```
// WRONG — no error shown when field is never touched
activityId: z.string().min(1, "Activity is required"),

// CORRECT — error shown even when field is never touched
activityId: z.string({ required_error: "Activity is required" }).min(1, "Activity is required"),
```

✗ Using `z.number()` for a Select field

```
// WRONG — Select returns strings, Zod number() will fail
activityId: z.number({ required_error: "Activity is required" }),

// CORRECT — store as string, convert to number in the submit handler
activityId: z.string({ required_error: "Activity is required" }).min(1, "..."),
// In onSubmit: activityId: Number(values.activityId)
```

✗ Not resetting the form when the modal re-opens

```
// WRONG — form keeps previous values when modal opens for a different item
useEffect(() => {
  reset(defaultValues);
}, [editCtx]); // missing `open` dependency
```

```
// CORRECT – reset whenever the modal opens OR the context changes
useEffect(() => {
  if (!open) return;
  reset(defaultValues);
}, [open, reset, defaultValues]);
```

✗ Calling `useFormContext` outside of `FormProvider`

```
// WRONG – ItemForm rendered without FormProvider
<ItemForm /> // useFormContext throws: "No FormProvider found"

// CORRECT – always wrap with FormProvider
<FormProvider {...methods}>
  <ItemForm />
</FormProvider>
```

Next: [Chapter 4 — Services](#)